Xinu (RAZ Extended Edition)
User's Manual

XFS filesystem, `cc` C compiler, `abclc` actor language, `wm` window manager

---

This manual describes an extended Embedded Xinu that adds a hierarchical filesystem **XFS** (on a RAM disk), a tiny built-in C compiler **cc** and bytecode VM **a.out**, an actor-oriented small language **ABCL/c+** with its C translator **abclc**, a window manager **wm** (a framebuffer GUI on ARM/QEMU), and Unix-like shell extensions (`ls` / `cd` / `cat` / `make` / `edit`, …). All stock Embedded Xinu facilities (threads, semaphores, networking) remain available.

Contents

1 Introduction

Xinu is a small UNIX-like OS originally written by Douglas Comer for teaching. This extended edition adds a self-contained workbench that can edit, compile, and run C / ABCL source on the OS itself — so the whole develop-build-run loop closes inside QEMU (or on real hardware) without a host cross-compiler.

Platform	Use	Notes
`arm-qemu`	QEMU `-M versatilepb`	UART shell + framebuffer + WM auto-start
`arm-rpi`	Raspberry Pi hardware	UART / HDMI / USB keyboard

XFS / cc / abclc / make / edit behave identically on every platform; the WM-related pieces (`rotlines`, `wm_line`, …) are active on `arm-qemu` only.

2 Booting

2.1 Build

```
cd compile
make PLATFORM=arm-qemu        # default; arm-rpi also works
```

On success this produces `compile/xinu.boot`.

2.2 Run under QEMU

With `qemu-system-arm` installed, the bundled wrapper scripts are easiest.

Console-only mode (no framebuffer):

```
./compile/run-console.sh
```

The UART shell (`xsh$`) is multiplexed onto the terminal; quit with **Ctrl-A → x**. Env vars: `XINU_MEM` (default `128M`), `XINU_PLATFORM` (default `arm-qemu`).

Window mode (with the WM demo):

```
./compile/run-window.sh
```

The LCD framebuffer appears in a macOS Cocoa window; the UART shell runs in the launching terminal (`-serial mon:stdio` keeps the QEMU monitor alongside). Quit by closing the window or **Ctrl-A → x**. The window scales smoothly up to 4× by dragging a corner (zoom-to-fit).

2.3 Boot sequence

The `main()` thread in `system/main.c` does, in order:

1. print the OS version and memory layout;
2. `xfsBootstrap()` — format `RAMDISK0` as XFS and mount it at `/`;
3. seed `/home/hello.c`, `/home/sum.c`, `.../PingPong.abcl`, `.../RotLines.abcl`;
4. open the network device (when `NETHER` is enabled);
5. open CONSOLE / TTY1;
6. on `arm-qemu` only, `create()` the `wm_main` thread;
7. start a shell thread per CONSOLE / TTY1.

When boot finishes, the prompt `xsh$` means you can type commands.

3 Using the shell

3.1 Basics

```
command [args...]    [< input-redirect] [> output-redirect] [&]
```

`&` runs in the background; `< >` redirect to/from I/O devices; there is no completion or history.

3.2 Command list (added/extended by this edition)

Command	Usage	Description
`pwd`	`pwd`	Print the current directory
`cd`	`cd [PATH]`	Change directory (default `/`)
`ls`	`ls [-l] [PATH]`	Directory listing; `-l` shows type/size
`cat`	`cat FILE...`	Print file contents
`mkdir`	`mkdir DIR...`	Create directories
`rmdir`	`rmdir DIR...`	Remove empty directories
`touch`	`touch FILE...`	Create empty file / update mtime
`rm`	`rm FILE...`	Remove files
`cp`	`cp SRC DST`	Copy (DST truncated)
`mv`	`mv SRC DST`	Rename / move
`write`	`write FILE TEXT...`	Join args with spaces + newline into FILE
`edit`	`edit FILE`	Built-in Emacs-like editor
`mkfs`	`mkfs DEVICE [VOL]`	Format a block device as XFS
`mount`	`mount DEV MNT`	Mount an XFS
`umount`	`umount MNT`	Unmount an XFS
`cc`	`cc SRC [-o OUT]`	Compile C source to a.out
`abclc`	`abclc SRC.abcl [-o OUT]`	ABCL → C → a.out in one step
`make`	`make [TARGET...]`	Run the Makefile in the current directory
`run`	`run PATH`	Execute an a.out
`rotlines`	`rotlines [FRAMES] [DEG/F]`	Draw rotating lines on the WM
`clear`	`clear`	Clear the screen
`sleep`	`sleep N`	Sleep N ms

Implicit `run`. If you type a command the shell does not recognise, it tries `aoutRun(<that name>)` as a last resort (around `shell/shell.c:332`). So after `cc hello.c -o hello` you can launch the program directly by its path-like name (`xsh$ hello` → `hello...`).

4 XFS — the filesystem

A 4 KB-block hierarchical filesystem (`include/xfs.h`), created on a 16 MB RAM disk (`RAMDISK0`) at boot and mounted at `/`.

Item	Value
Block size	4096 B
Max filename length	56 B
inode size	128 B
Direct blocks	12 (= 48 KB)
Single indirect	1024 blocks (≈ 4 MB)
Double indirect	yes
Magic	`XFS!` (0x58465321)

Directories store `struct xdirent[]` (64 B/entry) inside the inode, read with `xfsReaddir()`. `xfsChdir()` / `xfsGetcwd()` keep a **per-thread** absolute-path CWD (`cd /home` in shell A does not affect shell B). `xfsBootstrap()` checks the first 64 KB of `RAMDISK0` for the XFS magic, runs `xfsMkfs` if absent, and mounts at `/`.

Main API from C:

```
int fd = xfsOpen("/home/foo", XFS_O_RDWR | XFS_O_CREAT | XFS_O_TRUNC);
xfsWrite(fd, buf, n);  xfsRead(fd, buf, n);
xfsSeek(fd, 0, XFS_SEEK_SET);  xfsClose(fd);
xfsMkdir("/home/sub");  xfsRename("/home/old", "/home/new");  xfsUnlink("/home/foo");
```

RAMDISK0 is RAM: files disappear when QEMU exits. Back up anything you want to keep on the host.

5 cc / run — the C compiler and launcher

5.1 Supported C subset

Documented at the top of `system/cc.c`: one source file, one `int main(void)` plus argument-less helpers; declarations `int x;` / `int x = expr;` only; statements `if/else`, `while`, `for`, `return`, blocks, expression statements; expressions over integers, strings, identifiers, `+ - * / %`, the comparisons, `! && ||` (logical), unary `-`, parentheses; only built-in calls; `#include` accepted and ignored. **Unsupported:** structs, pointers, arrays, floating point, multiple files.

5.2 Built-in functions

BI	Function	Use
0	`printf(fmt, ...)`	string / integer formatting
1	`puts(s)`	string + newline
2	`putchar(c)`	one character
3	`getchar()`	one character of input
4	`exit(code)`	terminate
5	`rgb(r,g,b)`	BGR565 16-bit colour
6	`wm_line(idx,x1,y1,x2,y2,color)`	set WM user-line slot (0..7)
7	`wm_render(on)`	enable/disable user-line rendering
8	`wm_clear()`	clear user lines
9	`sleep_ms(ms)`	sleep
10/11	`isin(deg)` / `icos(deg)`	Q12 fixed point (4096 = 1.0)
12/13	`screen_w()` / `screen_h()`	screen size

5.3 a.out format and shell usage

`struct aout_header` in `include/aout.h`:

```
+--------------------------+
| magic = "XAOU"           |
| version = 1              |
| code_size / const_size   |
| entry / nlocals          |
+--------------------------+
| bytecode body            |
| string constant pool     |
+--------------------------+
```

The VM is stack-based (256 entries) with one-byte opcodes (`OP_PUSH_I32`, `OP_LOAD_LOC`, `OP_ADD`, `OP_JZ`, `OP_CALL_BI`); see `aoutRun()` in `system/aout.c`.

```
xsh$ cc /home/hello.c -o /home/hello
xsh$ run /home/hello
hello...
xsh$ /home/hello              # implicit run also works
```

6 abclc — the ABCL/c+ actor language

ABCL/c+ is a small actor-oriented language ("classes + message sends"). Here it is two-stage: **abclc translates to C, and cc compiles that C to bytecode**.

```
class ClassName {
    var field1;
    method methodName(p1, p2) {
        field1 = p1 + 1;
        send other.methodName(arg1, arg2);
    }
}
main {
    new ClassName a;  new ClassName b;
```

```
    send a.methodName(b, 10);
}
```

Implicit identifiers `self` (current actor id) and `sender`; at most one `send` per method (tail-call style), so each actor processes one message at a time; all values are `int`-equivalent; limits: 4 classes, 16 methods/class, 8 fields, 8 args.

```
xsh$ cd /home/abclcp/abclc
xsh$ abclc PingPong.abcl
abclc: translated PingPong.abcl -> PingPong.c
abclc: compiled PingPong.c -> PingPong
xsh$ run PingPong
```

With `-o NAME`, both `NAME.c` (intermediate C) and `NAME` (a.out) are written. The emitted C uses only cc's built-ins plus a few scheduling helpers; one message runs per time-slice, and after a `send` the message is queued.

7 make — the minimal build system

`shell/xsh_make.c`: if the CWD has no `Makefile`, scan `*.c` and `*.abcl` and auto-generate one; read it and build each target in `TARGETS = ...` (or declaration order); `.c` → `cc SRC -o TARGET`, `.abcl` → `abclc SRC -o TARGET`.

```
# comment
TARGETS = hello sum PingPong RotLines
hello:    hello.c
sum:      sum.c
PingPong: PingPong.abcl
RotLines: RotLines.abcl
```

Targets with no explicit rule are inferred by checking `TARGET.c` then `TARGET.abcl`.

```
xsh$ cd /home
xsh$ make
make: generated Makefile (2 .c, 0 .abcl)
    cc hello.c -o hello
    cc sum.c -o sum
make: built 2 target(s)
xsh$ ./hello
hello...
```

8 edit — the built-in editor

`shell/xsh_edit.c` is an Emacs-like modeless editor (up to 400 lines × 256 chars). `edit /home/hello.c`; a non-existent path opens a new buffer.

Key	Action
C-f / C-b	forward / back one character
C-n / C-p	down / up one line
C-a / C-e	start / end of line
←↑→↓	arrows (ESC[sequences)
C-d	delete at cursor
Backspace	delete previous character
Enter	insert newline
Tab	two spaces
C-k	kill to end of line
C-l	redraw
C-x C-s	save
C-x C-c	save and exit
C-g	exit without saving

The status line shows `Lrow:Ccol  status`.

9 wm — the window manager

`apps/wm.c` drives, on QEMU's VersatilePB, the PL110 LCD controller (`0x10120000`, 16bpp BGR565) and the PL050 PS/2 mouse (`0x10007000`) directly, showing a desktop-like GUI on a 1024×1024 virtual screen (640×480 physical). With `run-window.sh`, `wm_main` starts at boot. It provides a status bar, three draggable demo windows, a mouse cursor, a user draw hook `wm_set_user_render(...)`, and direct drawing APIs (`put_pixel_pub`, `fill_rect_pub`, `rect_outline_pub`, `draw_string_pub`, `wm_draw_line`).

From C / ABCL the basic pattern is `rgb(r,g,b)`, `wm_render(1)`, `wm_line(idx,...)`, `wm_clear()`, `screen_w()`/`screen_h()` (see RotLines.abcl). The shell command `rotlines` is lower-level: it registers a per-frame draw function via `wm_set_user_render(rot_render)`.

10 Sample programs in detail

At boot, `system/main.c` seeds four files into XFS:

Path	Kind	Focus
`/home/hello.c`	C	`printf`
`/home/sum.c`	C	`for` loop
`.../PingPong.abcl`	ABCL	inter-actor messaging
`.../RotLines.abcl`	ABCL + WM	animation

10.1 hello.c and sum.c

```
int main(void) { printf("hello...\n"); return 0; }
```

`#include` is accepted/ignored; `printf` is built-in 0; the return value becomes `aoutRun()`'s result. The simplified bytecode:

```
ENTER 0                         ; main has no locals
PUSH_STR offset_of("hello...\n")
CALL_BI 0, 1                    ; printf, 1 arg
```

```
PUSH_I32 0
RET
```

`sum.c` uses a `for` loop (cc expands `for(init;cond;step)` to `init; while(cond){body; step;}`; note `+= ++ --` are unsupported):

```
xsh$ cc /home/sum.c -o /home/sum
xsh$ run /home/sum
sum 1..10 = 55
```

10.2 PingPong.abcl — inter-actor messaging

```
class Player {
    var hits;
    method bounce(other, n) {
        if (n > 0) {
            printf("Player %d: tick (n=%d) hits=%d\n", self, n, hits);
            hits = hits + 1;
            send other.bounce(self, n - 1);
        }
    }
}
main { new Player p1; new Player p2; send p1.bounce(p2, 6); }
```

`self` is the current actor id (creation order 0,1,…); `send` is asynchronous; when `n-1` reaches 0 no `send` is emitted and the chain stops:

```
Player 0: tick (n=6) hits=0
Player 1: tick (n=5) hits=0
Player 0: tick (n=4) hits=1
...
Player 1: tick (n=1) hits=2
```

10.3 RotLines.abcl — ABCL + WM animation

```
class Rotator {
    var angle;
    method spin(n) {
        if (n > 0) {
            wm_line(0, 512, 384,
                    512 + icos(angle) * 320 / 4096,
                    384 + isin(angle) * 320 / 4096, rgb(255, 80, 80));
            // slots 1..3 at +90/+180/+270 deg likewise
            sleep_ms(16);
            angle = angle + 3;
            send self.spin(n - 1);
        } else { wm_render(0); }
    }
    method start(n) { wm_render(1); send self.spin(n); }
}
main { new Rotator r; send r.start(360); }
```

`icos`/`isin` return Q12 fixed point (divide by 4096 after scaling by the radius); `sleep_ms(16)` ≈ 60 fps; sending to `self` forms the animation loop, and `wm_render(0)` removes the line layer at the end.

10.4 rotlines (shell command) and apps/abcl_program.c

`shell/xsh_rotlines.c` registers `rot_render()` with the WM via `wm_set_user_render()` and loops advancing the angle with `sleep(16)`; it draws with `wm_draw_line` (per-frame) rather than the slot-based `wm_line`. `apps/abcl_program.c` is a sample of the C that abclc *generates* (a bounded-buffer with `Buffer`/`Producer`/`Consumer`/`Controller` classes, mailbox + thread per actor, `dispatch_*` mega-switch); it shows the correspondence ABCL `class` → C `dispatch_<Class>`, `field` → `enum` index, `send` → `enqueue`.

11 Development workflow

```
$ ./compile/run-window.sh
xsh$ cd /home && make && ./hello && ./sum
xsh$ edit /home/myprog.c          # C-x C-s save, C-x C-c exit
xsh$ cc myprog.c -o myprog && ./myprog
xsh$ cd /home/abclcp/abclc
xsh$ edit MyActor.abcl && abclc MyActor.abcl && ./MyActor
xsh$ rotlines
```

Tips: build one target with `make MyActor`; rebuild all with `rm Makefile && make`; XFS has up to 32768 inodes, so avoid creating many files.

12 Troubleshooting

Symptom	Cause / fix
`qemu-system-arm not found`	`brew install qemu` / `apt install qemu-system-arm`
stale build boots	`make clean && make PLATFORM=arm-qemu` in `compile/`
no shell prompt	`Ctrl-A` → `c` for the QEMU monitor, then `info registers`
`cc: line N: ...` error	unsupported syntax (structs, pointers, arrays, float, …)
`abclc: translation failed`	two `send`s in one method, too many `var`s / methods
`command not found` but file exists	implicit `run` takes a single path token; use `run PATH ARGS...`
no WM window	is it an `arm-qemu` build? did you use `run-window.sh`?
files vanished	the RAM disk is volatile — back up on the host
overlong path	`XFS_PATH_MAX = 256`; longer paths are truncated
garbled screen	`C-l` in the editor; `clear` in the shell

Appendix A. File map

Area	Key files
Boot	`system/main.c`, `system/initialize.c`, `compile/run-*.sh`
XFS	`include/xfs.h`, `system/xfs.c`, `device/ramdisk/`
cc	`include/aout.h`, `system/cc.c`, `system/aout.c`, `shell/xsh_cc.c`
abclc	`include/abclc.h`, `system/abclc.c`, `shell/xsh_abclc.c`
make/edit	`shell/xsh_make.c`, `shell/xsh_edit.c`
WM	`apps/wm.c`, `apps/abcl_xinu_gui.c`, `shell/xsh_rotlines.c`
platform	`compile/platforms/arm-qemu/`, `compile/platforms/arm-rpi/`

Appendix B. Existing samples (after FS seeding)

```
/home/
├── hello.c
├── sum.c
└── abclcp/
    └── abclc/
        ├── PingPong.abcl
        └── RotLines.abcl
```

`cd /home && make` builds `hello`, `sum`; `cd /home/abclcp/abclc && make` builds `PingPong`, `RotLines`. To go deeper, read `system/cc.c`, `system/aout.c` (the bytecode VM), and `system/abclc.c` (the ABCL translator).